

ShopSC Premium

Full-Stack MERN Luxury E-Commerce Architecture & API Documentation

1. Executive Project Overview

ShopSC Premium (internally mapped to the **ShopEZ** infrastructure) is a high-performance, modern full-stack web application engineered using the MERN ecosystem (MongoDB, Express.js, React, Node.js). The platform is stylized to represent a top-tier luxury storefront offering ultra-premium technology catalogs and bespoke couture items starting from ₹10,000 to ₹1,99,999+.

The backend engine utilizes robust asynchronous routing models, unified schema declarations, and an enterprise hybrid deletion protocol to maintain absolute state synchronization between local React client sessions and the core database.

Core Tech Stack Specifications:

- **Frontend Matrix:** React (Vite environment), Modern Functional Components, Inline-styled UI controls.
- **Backend Layer:** Node.js runtime, Express.js micro-framework, asynchronous controller wrappers.
- **Persistence Tier:** MongoDB Community Server instance managed natively through ODM Mongoose layers.
- **Syntax Target:** ECMAScript Modules (ESM) using pure **import/export** runtime patterns.

2. System Architecture & File Topology

The codebase utilizes an elegant separation of concerns across dedicated folders. The application root splits explicitly into isolated execution scopes for front-end presentation and server configurations.

Backend Structural Blueprint

```
server/  
├── models/  
│   └── Schemas.js (Unified Mongoose definitions for Products & Carts)  
├── routes/  
│   ├── cart.js (Core dynamic cart CRUD execution engine)  
│   └── shopRoutes.js (Product management & catalog APIs)
```

- └─  index.js (Primary system entrypoint and application listener)
- └─  seed.js (Luxury catalog database seeder script)
- └─  package.json (Dependency mapping & metadata configuration)
- └─  s.env (Environment runtime variables configuration)

3. Backend Database Architecture & Implementation

The backend combines global application middleware (CORS handling, unified JSON line mapping) directly with isolated route modules dynamically bound under a universal global path prefix `/api`.

Server Entrypoint (`server/index.js`)

The centralized configuration hooks up dynamic route routers and initializes cross-origin validation:

```

import express from 'express';
import mongoose from 'mongoose';
import cors from 'cors';
import dotenv from 'dotenv';
import shopRoutes from './routes/shopRoutes.js';
import cartRoutes from './routes/cart.js';

dotenv.config();
const app = express();

// Application Global Middleware Matrix
app.use(express.json());
app.use(cors());

// Universal API Subrouting Assignment
app.use('/api', shopRoutes);
app.use('/api', cartRoutes);

app.get('/', (req, res) => {
  res.send("ShopEZ Backend Server is Running Successfully!");
});

const PORT = process.env.PORT || 8000;
const mongoURI = "mongodb://127.0.0.1:27017/ShopEZ";

mongoose.connect(mongoURI)
  .then(() => {
    console.log("Connected to your MongoDB database successfully.");
    app.listen(PORT, () => {
      console.log(`App server is running on port ${PORT}`);
    });
  })
  .catch((error) => {
    console.error("Database connection failed:", error.message);
  });

```

4. REST API Documentation & Endpoints

All communications with the data tier execute over structured asynchronous HTTP transactions. The backend explicitly guarantees standard operational response protocols.

Method	Endpoint Route	Purpose / Behavioral Context	Payload Spec
GET	/api/products	Retrieves the current premium catalog array from MongoDB.	None
POST	/api/cart/add	Appends a product mapping into the user's active persistent bag.	JSON object
DELETE	/api/cart/ remove/:id	Removes an item cleanly using a dual ObjectId/Title pattern fallback.	URL Parameter

Dynamic Cart Router Engine (`server/routes/cart.js`)

```
import express from 'express';
const router = express.Router();
import { Cart } from '../models/Schemas.js';

// 🗑️ DELETE ROUTE: Drops specific document from client application state view
router.delete('/cart/remove/:id', async (req, res) => {
  try {
    const itemId = req.params.id;
    let deletedItem = null;

    // Strategy A: Direct MongoDB native hex verification sequence
    if (itemId.match(/^([0-9a-fA-F]{24})$/)) {
      deletedItem = await Cart.findByIdAndDelete(itemId);
    }

    // Strategy B: Fallback string matching block to flush out persistent legacy
    views
    if (!deletedItem) {
      deletedItem = await Cart.deleteOne({ title: itemId });
    }

    res.status(200).json({ message: "Item successfully removed from cart! ✨" });
  } catch (err) {
    console.error("Error deleting cart item:", err);
    res.status(500).json({ error: "Internal server error. Could not drop item."
    😞" });
  }
});

export default router;
```

5. Database Automation & Luxury Seeding Strategy

To preserve UI display consistency, old boilerplate elements are cleared automatically upon running the administrative database deployment script. The architecture provides zero-downtime flushing vectors to swap standard entries for elite pricing distributions smoothly.

Seeder Initialization Execution Paradigm:

To trigger data synchronization from the root scope folder, execute: `node server/seed.js` or navigate into directory paths and run:

```
cd server
node seed.js
```

Complete Data Injector Script (`server/seed.js`)

```

import mongoose from 'mongoose';
import { Product, Cart } from './models/Schemas.js';

const MONGO_URI = "mongodb://127.0.0.1:27017/ShopEZ";

const mockProducts = [
  {
    title: "Apple iPhone 16 Pro Max",
    description: "Titanium design, A18 Pro chip, Camera Control, and monumental battery life.",
    price: 199999,
    mainimg: "https://images.unsplash.com/photo-1695048133142-1a20484d2569?w=500&auto=format&fit=crop&q=60"
  },
  {
    title: "Samsung Galaxy S26 Ultra",
    description: "Built-in S Pen, 200MP ultimate camera setup, and extreme Galaxy AI optimization.",
    price: 129999,
    mainimg: "https://images.unsplash.com/photo-1610945265064-0e34e5519bbf?w=500&auto=format&fit=crop&q=60"
  },
  {
    title: "Boys Savile Row Bespoke Suit Set",
    description: "Sharp bespoke 3-piece formal evening suit tuxedo, masterfully tailored using 100% fine Merino virgin wool.",
    price: 45999,
    mainimg: "https://images.unsplash.com/photo-1519238263530-99bdd11df2ea?w=500&auto=format&fit=crop&q=60"
  },
  {
    title: "Girls Royal Pastel Silk Gown",
    description: "Elegant layered silk tulle mesh party gown, individually hand-decorated with Swarovski crystal floral embroidery.",
    price: 38000,
    mainimg: "https://images.unsplash.com/photo-1518831959646-742c3a14ebf7?w=500&auto=format&fit=crop&q=60"
  }
];

const seedDatabase = async () => {
  try {
    await mongoose.connect(MONGO_URI);
    console.log("Connected to database for seeding luxury collection...");

    // Drop any previous instances in user records to block state corruption
    if (Cart) {
      await Cart.deleteMany({});
    } else {

```

```

        await mongoose.connection.collection('carts').deleteMany({});
    }
    console.log("🧹 Stale/glitched shopping bags emptied successfully!");

    await Product.deleteMany({});
    await Product.insertMany(mockProducts);
    console.log("🚀 Elite Million-Tier Catalog Seeded Successfully with Premium
Luxury Goods!");

    mongoose.connection.close();
  } catch (error) {
    console.error("Seeding failed:", error);
  }
};

seedDatabase();

```

6. System Operational Guide

To execute the full-stack suite concurrently, ensure your local MongoDB daemon is operating locally on port **27017** . Open two micro-terminal sessions running the following commands:

Micro-Terminal Session 1: Node Engine Service Deployment

```

cd server
node index.js

```

Expect logging signature confirmation: **App server is running on port 8000**

Micro-Terminal Session 2: React UI Core Bundle Compilation

```

cd client
npm run dev

```

Launch target workspace local link inside user web browser: **<http://localhost:5173/>**